

Jeu de la Vie

Documentation Technique & Guide

Implémentation Java · 5 Design Patterns · Interface Swing

Réalisé par KONG Chaosok

1. Présentation du projet

Ce projet est une implémentation en Java du célèbre automate cellulaire « Jeu de la Vie » inventé par le mathématicien John Horton Conway en 1970. Il illustre cinq design patterns du GoF (Gang of Four) dans un contexte concret, tout en offrant une interface graphique complète sous Swing.

Objectif pédagogique

Démontrer l'application conjointe des patterns État, Singleton, Observateur, Commande et Visiteur dans un automate cellulaire interactif.

2. Structure du projet

L'arborescence ci-dessous liste tous les fichiers sources et leur responsabilité :

Fichier	Rôle
CelluleEtat.java	Interface du pattern État — déclare vit(), meurt(), estVivante(), accepte()
CelluleEtatVivant.java	État Vivant (Singleton) — délègue l'action meurt() et dirige le Visiteur
CelluleEtatMort.java	État Mort (Singleton) — délègue l'action vit() et dirige le Visiteur
Cellule.java	Entité principale — mémorise (x, y, état) et délègue tout à l'état courant
Commande.java	Classe abstraite du pattern Commande — déclare executer() et le récepteur
CommandeVit.java	Commande concrète : donne la vie à une cellule
CommandeMeurt.java	Commande concrète : fait mourir une cellule
Visiteur.java	Classe abstraite du pattern Visiteur — deux méthodes de visite
VisiteurClassique.java	Règles de Conway (B3/S23)
VisiteurHighLife.java	Règles HighLife (B36/S23)
VisiteurDayNight.java	Règles Day & Night (B3678/S34678)
Observateur.java	Interface du pattern Observateur — déclare actualise()
Observable.java	Classe abstraite — gère la liste d'observateurs et la notification
ObservateurConsole.java	Observateur texte — affiche génération et nombre de vivants en console
JeuDeLaVie.java	Modèle principal (Observable) — grille, commandes, visiteur, structures prédéfinies

3. Design Patterns utilisés

3.1 Pattern État (State)

Chaque cellule délègue entièrement son comportement (vivre, mourir, accepter un visiteur) à son objet état courant. Cela évite les blocs if/else sur un booléen « vivant » et rend l'ajout d'un nouvel état trivial.

Classe / Interface	Rôle dans le pattern
CelluleEtat	Contexte (interface) — contrat commun à tous les états
CelluleEtatVivant	État concret Vivant — vit() ne fait rien, meurt() change l'état
CelluleEtatMort	État concret Mort — meurt() ne fait rien, vit() change l'état
Cellule	Contexte — possède une référence à CelluleEtat et délègue

Extrait clé — délégation dans Cellule.java :

```
public void vit()    { etat.vit(this);    }
public void meurt() { etat.meurt(this); }
public boolean estVivante() { return etat.estVivante(); }
```

3.2 Pattern Singleton

Les deux états sont des Singletons : une seule instance de CelluleEtatVivant et une seule instance de CelluleEtatMort existent pour toute la durée de vie du programme. Cela économise la mémoire sur une grille de plusieurs milliers de cellules.

```
private static CelluleEtatVivant instance = new CelluleEtatVivant();
private CelluleEtatVivant() {} // Constructeur privé
public static CelluleEtatVivant getInstance() { return instance; }
```

Avantage

Sur une grille 70×70 (4 900 cellules), sans Singleton on créerait jusqu'à 4 900 objets état redondants. Avec Singleton : exactement 2 objets état pour toute la simulation.

3.3 Pattern Observateur (Observer)

JeuDeLaVie est l'Observable (le sujet). JeuDeLaVieUI et ObservateurConsole sont des Observateurs. À chaque génération ou réinitialisation, le modèle appelle notifieObservateurs() — toutes les vues se mettent à jour automatiquement sans couplage direct.

Classe	Rôle
Observable	Classe abstraite — liste d'observateurs, attache/détache/notifie
Observateur	Interface — contrat actualise()
JeuDeLaVie	Sujet (extends Observable) — notifie après chaque génération

JeuDeLaVieUI	Observateur graphique — redessine la grille Swing
ObservateurConsole	Observateur texte — affiche les stats en console

3.4 Pattern Commande (Command)

Plutôt que de modifier la grille pendant l'analyse (ce qui fausserait les calculs des voisines), les décisions de vie/mort sont stockées dans une file de Commandes. Toutes les commandes sont exécutées en bloc une fois l'analyse terminée.

Classe	Rôle
Commande	Classe abstraite — déclare <code>executer()</code> et le récepteur (<code>Cellule</code>)
CommandeVit	Commande concrète — appelle <code>cellule.vit()</code>
CommandeMeurt	Commande concrète — appelle <code>cellule.meurt()</code>
JeuDeLaVie	Invocateur — maintient la <code>List<Commande></code> et appelle <code>executeCommandes()</code>
Cellule	Récepteur — effectue réellement le changement d'état

Séquence par génération dans `JeuDeLaVie.calculerGenerationSuivante()` :

- `distribueVisiteur()` → chaque cellule accepte le visiteur qui crée des Commandes
- `executeCommandes()` → toutes les commandes sont jouées en une passe
- `notifieObservateurs()` → les vues sont rafraîchies

3.5 Pattern Visiteur (Visitor)

Les règles du jeu sont entièrement découplées de la structure des cellules. Changer de règles ne nécessite pas de toucher à `Cellule` ou `CelluleEtat` — il suffit de passer un autre Visiteur.

Classe	Règles
VisiteurClassique	Conway B3/S23 — naissance si 3 voisines, survie si 2 ou 3
VisiteurHighLife	HighLife B36/S23 — naissance si 3 ou 6 voisines, survie si 2 ou 3
VisiteurDayNight	Day & Night B3678/S34678 — survie 3,4,6,7,8 / naissance 3,6,7,8

4. Documentation détaillée des classes

Cellule.java

Responsabilité

Entité de base de la grille. Mémorise sa position (x, y) et son état courant. Toutes les actions sont déléguées à `CelluleEtat`.

Attributs

- `etat` : `CelluleEtat` — l'état courant (vivant ou mort)
- `x`, `y` : `int` — coordonnées dans la grille

Méthodes principales

Méthode	Description
<code>vit()</code>	Délègue à <code>etat.vit(this)</code> — peut changer l'état en Vivant
<code>meurt()</code>	Délègue à <code>etat.meurt(this)</code> — peut changer l'état en Mort
<code>estVivante()</code>	Retourne <code>etat.estVivante()</code> — true si la cellule est en état Vivant
<code>accepte(Visiteur v)</code>	Délègue à <code>etat.accepte(v, this)</code> — dirige vers <code>visiteCelluleVivante</code> ou <code>visiteCelluleMorte</code>
<code>nombreVoisinesVivantes(jeu)</code>	Compte les 8 voisins (Moore) en ignorant les bords hors-grille

JeuDeLaVie.java

Responsabilité

Modèle central (MVC). Gère la grille 2D, la file de commandes, le visiteur actif et les structures prédéfinies. Étend `Observable` pour notifier les vues.

Attributs

- `grille` : `Cellule[][]` — tableau 2D de toutes les cellules
- `xMax`, `yMax` : `int` — dimensions de la grille
- `commandes` : `List<Commande>` — file des actions en attente
- `visiteur` : `Visiteur` — règles actives (classique, HighLife ou Day & Night)

Méthodes principales

Méthode	Description
<code>calculerGenerationSuivante()</code>	Orchestre un cycle : analyse → commandes → notification
<code>reinitialiserGrille(densite)</code>	Regénère la grille aléatoirement avec la densité donnée (0.0–1.0)
<code>distribueVisiteur()</code>	Fait accepter le visiteur à chaque cellule de la grille
<code>executeCommandes()</code>	Exécute toutes les commandes en attente, puis vide la liste
<code>ajouterCommande(c)</code>	Ajoute une <code>Commande</code> à la file (appelé par les <code>Visiteurs</code>)
<code>setVisiteur(v)</code>	Change les règles actives à la volée
<code>chargerPlaneur()</code>	Charge le glider de Conway (5 cellules)
<code>chargerClignotant()</code>	Charge un oscillateur de période 2 (3 cellules en ligne)

chargerVaisseauLeger()	Charge le LWSS (vaisseau léger se déplaçant horizontalement)
chargerCanonAPlaneurs()	Charge le canon de Gosper (tire un planeur toutes les 30 générations)
redimensionner(x, y)	Recrée la grille à la nouvelle taille et la réinitialise à 30%
getGrilleXY(x, y)	Retourne la cellule à (x, y) ou null si hors-grille

JeuDeLaVieUI.java

Responsabilité

Vue Swing (implémente Observateur). Affiche la grille, gère les interactions utilisateur (boutons, sliders, combos, clic/drag sur la grille) et dessine le HUD en overlay.

Composants de l'interface

Contrôle	Action
► Commencer / Pause	Lance ou stoppe la boucle de simulation (Timer Swing)
» Suivant	Avance d'une seule génération et incrémente le compteur
↺ Réinitialiser	Regénère une grille aléatoire à 30% de densité
Combo Structures	Charge un pattern prédéfini (planeur, clignotant, vaisseau, canon)
 - /  +	Zoom arrière / avant (taille cellule en pixels)
↕ Ajuster	Réinitialise le zoom pour que la grille remplisse la fenêtre
Slider Vitesse	Délai du Timer : 50 ms (rapide) → 1000 ms (lent)
Slider Densité	Densité de la réinitialisation aléatoire (0% → 100%)
Combo Règles	Bascule entre Conway, HighLife et Day & Night
Couleur Cellules Vivantes	Ouvre un sélecteur de couleur pour les cellules vivantes
Couleur Cellules Mortes	Ouvre un sélecteur de couleur pour le fond
Clic sur la grille	Bascule l'état d'une cellule (vivante ↔ morte)
Drag sur la grille	Trace des cellules vivantes en faisant glisser la souris

HUD (Heads-Up Display)

Un panneau semi-transparent (fond noir à 70% d'opacité) est dessiné en overlay dans la méthode `paintComponent()` de `ZoneGrille`. Il suit le scroll et affiche en temps réel : numéro de génération, nombre de cellules vivantes, vitesse ou statut « En pause ».

Observable.java & Observateur.java

Observable est une classe abstraite qui gère la liste d'observateurs. Elle expose `attacheObservateur()`, `detacheObservateur()` et `notifieObservateurs()`. Observateur est une interface à méthode unique : `actualise()`, que chaque vue doit implémenter.

ObservateurConsole.java

Observateur secondaire qui affiche dans la console standard (stdout) le numéro de génération et le nombre de cellules vivantes à chaque appel à `actualise()`. Utile pour le débogage sans interface graphique.

Commande.java, CommandeVit.java, CommandeMeurt.java

Commande est une classe abstraite qui déclare la méthode `executer()` et l'attribut protégé `cellule` (le récepteur). `CommandeVit` appelle `cellule.vit()` et `CommandeMeurt` appelle `cellule.meurt()`. Ces objets sont créés par les Visiteurs et empilés dans `JeuDeLaVie.commandes`.

Visiteur.java, VisiteurClassique.java, VisiteurHighLife.java, VisiteurDayNight.java

Chaque visiteur concret implémente deux méthodes :

- `visiteCelluleVivante(Cellule c)` — décide si la cellule doit mourir
- `visiteCelluleMorte(Cellule c)` — décide si la cellule doit naître

Les règles de chaque variante sont résumées ci-dessous :

Visiteur	Survie (S)	Naissance (B)
VisiteurClassique (Conway)	2, 3 voisines	3 voisines
VisiteurHighLife	2, 3 voisines	3 ou 6 voisines
VisiteurDayNight	3, 4, 6, 7 ou 8 voisines	3, 6, 7 ou 8 voisines

5. Flux d'exécution

5.1 Démarrage (main)

- Instanciation de `JeuDeLaVie(70, 70)` → grille 70×70 initialisée à 50%
- Instanciation de `JeuDeLaVieUI` → fenêtre Swing créée et affichée
- Instanciation de `ObservateurConsole` → sortie console activée
- `jeu.attacheObservateur(ui)` et `jeu.attacheObservateur(console)`
- `jeu.notifieObservateurs()` → premier affichage

5.2 Cycle par génération

Étape	Classe / Méthode	Action
1	Timer (<code>JeuDeLaVieUI</code>)	Déclenche <code>avancerGeneration()</code> toutes les N ms
2	<code>JeuDeLaVie.distribueVisiteur()</code>	Pour chaque cellule, appelle <code>cellule.accepte(visiteur)</code>
3	<code>CelluleEtat.accepte()</code>	Dirige vers <code>visiteCelluleVivante</code> ou <code>visiteCelluleMorte</code>

4	Visiteur.visiteXxx()	Crée CommandeVit ou CommandeMeurt si besoin
5	JeuDeLaVie.executeCommandes()	Applique tous les changements d'état en une seule passe
6	JeuDeLaVie.notifieObservateurs()	Appelle actualise() sur chaque observateur
7	JeuDeLaVieUI.actualise()	Redessine la grille via SwingUtilities.invokeLater()
8	ObservateurConsole.actualise()	Affiche stats génération en console

6. Compilation et lancement

Depuis le dossier racine du projet (qui contient le dossier src/) :

```
mkdir bin
javac -d bin src/*.java
java -cp bin JeuDeLaVie
```

Avec Java 17+ :

```
javac --release 17 -d bin src/*.java
java -cp bin JeuDeLaVie
```

Prérequis

Java 11 minimum. Java 17 LTS recommandé. Aucune dépendance externe — la bibliothèque Swing est incluse dans le JDK.

7. Synthèse des Design Patterns

Pattern	Classes	Problème résolu
État	CelluleEtat, CelluleEtatVivant, CelluleEtatMort, Cellule	Évite les if/else sur l'état — comportement délégué à l'objet état
Singleton	CelluleEtatVivant, CelluleEtatMort	Une seule instance par état — économie mémoire sur la grille entière
Observateur	Observable, Observateur, JeuDeLaVie, JeuDeLaVieUI, ObservateurConsole	Découple le modèle des vues — toutes les vues se mettent à jour automatiquement
Commande	Commande, CommandeVit, CommandeMeurt, JeuDeLaVie, Cellule	Évite la modification en cours d'analyse — file d'actions exécutée en une passe
Visiteur	Visiteur, VisiteurClassique, VisiteurHighLife, VisiteurDayNight	Règles découplées des cellules — changer de règles sans toucher aux cellules

8. Pistes d'amélioration

- Grille torique : connecter les bords opposés pour éviter les effets de bord
- Export image/GIF : sauvegarder l'état de la grille comme image PNG ou animation GIF
- Undo/Redo : exploiter la file de Commandes pour implémenter un historique annulable
- Grille dynamique : agrandir automatiquement la grille quand des structures atteignent les bords
- Nouveau visiteur : ajouter Brain's Brain, Life without Death ou d'autres règles célèbres
- Tests unitaires : JUnit 5 sur JeuDeLaVie et les Visiteurs pour valider les règles